

Foolproof iOS Clone Prompts Playbook

Copy/paste prompts that ship an Expo + FastAPI iOS app to TestFlight with zero rebuild loops.

HOW TO USE

- You are about to clone an iOS app and ship it to TestFlight.
- You don't want a 20-build, 20-credit loop like the last time.
- You want to read three pages, copy, paste, walk away.
- This playbook is built for exactly that.

BASED ON Builds #11 → #20 of Satoshi Cloud Miner. We made every mistake so you do not have to. The prompts below would have collapsed that journey into 1-2 EAS builds.

0 · QUICKSTART

How to use this playbook

This document contains **10 numbered prompts + 1 master prompt**. If you follow them in order, you will skip every painful rebuild loop we hit on the Satoshi Cloud Miner project (App Store Connect IAP mismatches, icon white-borders, simulated data, iPad screenshot rejections, tracking-permission rejections, etc.).

The three rules

1 · Send the Master Prompt first.	It declares every constraint upfront. Without it, the agent invents assumptions that cost you credits.
2 · Send each follow-up prompt in order.	Each one assumes the previous step is done. Do not skip ahead — they are sequential.
3 · Replace every <...> placeholder.	Wherever you see angle brackets, paste your real value. If you can't fill one, ask the agent to source it for you in the same chat.

Before you paste anything — collect these credentials

Item	Where to get it	Placeholder you'll replace
Apple Team ID	https://developer.apple.com → Membership	<TEAM_ID>
ASC App Manager API key (.p8)	App Store Connect → Users & Access → Integrations → App Store Connect API	<ASC_ISSUER_ID> / <ASC_P8>
In-App Purchase Server API key (.p8)	App Store Connect → Users & Access → Integrations → In-App Purchase	<IAP_KEY_ID> / <IAP_P8>
Expo personal access token	https://expo.dev → account settings → access tokens	<EXPO_TOKEN>
Existing ASC app record	App Store Connect → Apps	<BUNDLE_ID> / <ASC_APP_ID>
LLM key	Either Emergent universal LLM key (recommended), or your OpenAI/Anthropic/Gemini key	<LLM_KEY>
Payment / wallet keys	Whatever vendor (Stripe, Blink, BTCPay, Twilio...)	<VENDOR_KEY>

1 · THE MASTER PROMPT

Send this one prompt FIRST. Always.

This is the single most important message in the whole playbook. Send it as the very first message in a fresh chat. Every gotcha that burned credits on the Satoshi Cloud Miner project is preempted by a constraint in this prompt.

WHY: Without this prompt, the agent will assume sensible defaults that don't match Apple's reality (e.g. iPad screenshots required, hardcoded BTC rate, AI-generated icon with white border).

■ PROMPT 00 THE MASTER PROMPT

GOAL

Clone <SOURCE APP NAME + URL OR SCREENSHOTS> into a production-ready iOS app called <MY APP NAME> and ship it to TestFlight + the App Store. I will NOT review interim builds – deliver the final result. You decide every implementation detail; I only weigh in on product behavior or branding.

MY CREDENTIALS (use these, do not ask later)

Apple Team ID: <TEAM_ID>
ASC API key: <ASC_KEY_ID> / <ASC_ISSUER_ID> (.p8 attached)
IAP Server API key: <IAP_KEY_ID> (.p8 attached)
EXPO_TOKEN: <EXPO_TOKEN>
ASC app record: bundle=<BUNDLE_ID> app_id=<ASC_APP_ID>
LLM key: Use the Emergent universal LLM key
Payment/wallet keys: <VENDOR_KEYS_HERE>

NON-NEGOTIABLE CONSTRAINTS – fail the build if any is violated

1. No simulated data. No hardcoded prices, fake rates, deterministic-random AI, or MOCK_* constants. If a real source is unavailable, surface 'unavailable' in the UI rather than fake it.
2. App Store Connect is the SOURCE OF TRUTH for IAP product IDs. Enumerate ASC via API BEFORE writing any IAP code. Never invent a product_id in the backend that doesn't already exist in ASC.
3. App icon: 1024x1024 PNG, RGB (no alpha), full-bleed dark background, no white border, no transparent corners. Programmatically sample edge pixels before committing.
4. AI-generated assets must have correct magic bytes. If filename says .png, file MUST start with 89 50 4E 47.
5. supportsTablet: false by default (otherwise Apple requires iPad screenshots).
6. Do NOT declare iOS permissions you don't use. No NSUserTrackingUsageDescription unless an SDK actually requests tracking – otherwise Apple forces the App Privacy questionnaire.
7. Quality gates before EVERY EAS build: npx expo-doctor (must be N/N), yarn tsc --noEmit, backend tests. Block the build on any failure.
8. Use the App Store Connect API to upload everything – description, keywords, screenshots, preview video, IAP linkage, review submission. Do NOT write me a 'manual upload guide'.

DEFINITION OF DONE

- App Store Connect shows WAITING_FOR_REVIEW with all IAPs auto-bundled.
- Backend regression tests pass.
- A persistent auto_ship watcher is armed for v1.0.1 so I never need to manually ship a follow-up build.

Confirm you understand every constraint before writing code. Push back on anything that needs Apple policy exceptions (gambling, crypto withdrawals, age-gated content). Then proceed end-to-end.

2 · THE 7 GOTCHAS

Failures we hit — and the one sentence that would have prevented each

Add ALL seven lines to the bottom of the Master Prompt if you want belt-and-suspenders coverage.

#	What went wrong on Satoshi Cloud Miner	Sentence to add to prompt
1	starter_099 IAP existed in backend code but Apple never had it	Before EveryTestFlight deployment, enumerate App Store Connect IAPs via the v2 API and update starter_099 IAP existence in backend code
2	BTC_USD_RATE = 65000.0 hardcoded as a 'simulated rate' for 10 weeks	Any price if from a live API with a fallback cascade. No magic numbers
3	AI Trading Agents were deterministic random — the UI said 'AI' but the data showed 'Choice'	Anywhere the data shows an AI/Choice, it must be a real LLM/API call with caching
4	AI-generated icon was JPEG bytes saved as .png → EAS build failed	Before build, verify the file's magic bytes match the extension. Re-export if mismatch
5	Icon had a 69-px white border baked in → visible white ring	Sample the app icon on any 1024x1024 icon; if pixels >235,>235,>235 are found, crop
6	supportsTablet: true + unused NSUserTrackingUsageDescription	Default supportsTablet is false. Never add a permission string unless I explicitly request it
7	Apple's 'first IAP must ship with first version' + screenshot	Is screenshot SUBMITTED FOR REVIEW (via App Store Connect). Upload all metadata

3 · THE 10 NUMBERED PROMPTS

Send these in order, one per message

Each prompt assumes the previous one finished cleanly. Wait for the agent to confirm success (a green log line, a screenshot, or an HTTP 200 response) before sending the next one.

■ PROMPT 01 Lock in the product spec

Here is my `PRODUCT_SPEC.md` (attached). Read every section and confirm you understand it. Specifically confirm:

- the exact list of screens
- every IAP tier with `product_id`, type, price
- every external integration with key scopes
- every iOS permission with the code path that uses it

Push back on anything that requires an Apple App Store policy exception. Propose alternatives where needed. Reply with a TL;DR of the spec in your own words so I know we are aligned.

■ PROMPT 02 Build the backend + Expo skeleton (no UI yet)

Build the FastAPI backend + Expo Router skeleton.

- Auth: email+password + JWT, with `/api/auth/{register,login,me}`.
- All data models from the spec, with MongoDB indexes.
- Every API endpoint stubbed with real schema responses (200 OK).
- expo-router wired with `app/(tabs)/_layout.tsx`, but tabs render placeholder Text only – no real UI yet.

Then run the `deep_testing_backend_v2` agent and paste the report. Do NOT proceed to UI until backend regression is green.

■ PROMPT 03 Bootstrap App Store Connect (BEFORE any IAP code)

Using my ASC App Manager key, do this end-to-end via the ASC API:

1. Verify the existing app record `<ASC_APP_ID>`.
2. Create every IAP from `PRODUCT_SPEC.md` with the correct type (consumable / non-consumable / subscription), localized name, price tier, and review screenshot (use a 1290x2796 placeholder PNG if needed).
3. Move each IAP to `READY_TO_SUBMIT` state.

After running, print the FINAL list of product IDs that exist in ASC. These are the source of truth for the backend `SHOP_PACKAGES` table. If any spec IAP cannot be created via API (e.g. missing review screenshot), tell me explicitly so we remove it from the spec.

■ PROMPT 04 Wire integrations one at a time, prove each is live

For each integration in my spec, do this loop:

1. Write the integration module (one file in backend/integrations/).
2. Add a /api/diag/<integration_name> endpoint that performs ONE real round-trip to the vendor (e.g. fetch BTC rate, validate a test Apple receipt, send a \$0.001 Lightning ping).
3. Hit it and paste the JSON response in your reply.

Do NOT move to the next integration until the diag endpoint returns a real, non-mocked response. No 'MOCKED' values anywhere.

■ PROMPT 05 Build the UI from the spec

Build every screen from PRODUCT_SPEC.md.

- After EVERY screen, run yarn tsc --noEmit and npx expo-doctor.
- Capture a screenshot of the rendered screen from the web preview (<http://localhost:3000>) at iPhone 14 Pro dimensions and paste it.
- Use real data from the integrations wired in Prompt 04 – no placeholder text, no Lorem Ipsum, no MOCK_*.

If any check fails, stop and fix that screen before moving on.
Do not pile up TODOs.

■ PROMPT 06 Generate marketing assets — with pixel-level verification

Generate all App Store marketing assets in ONE batch:

- App icon: 1024x1024 PNG, RGB only (no alpha), brand-colored full-bleed background, glyph centered, NO white border.
- 12 screenshots: 1290x2796 (iPhone 6.7), 1242x2688 (6.5), 1242x2208 (5.5).
- 1 App Preview video: 1080x1920 H.264, 15-30s, STEREO or SILENT audio only (no mono – Apple's encoder rejects MOV_RESAVE_STEREO).

Then run a verification pass that PROVES:

- icon edge pixels (sample 10 corners at <20px from edge) are NOT white
- every PNG starts with bytes 89 50 4E 47
- video probe (ffprobe) shows acodec=aac stereo OR no audio stream

Paste the pixel-sample report. Re-encode if anything fails.

■ PROMPT 07 Harden app.json for Apple submission

Update /app/frontend/app.json with these EXACT settings before any

EAS build:

- ios.supportsTablet: false
- ios.config.usesNonExemptEncryption: false
- ios.infoPlist.ITSAppUsesNonExemptEncryption: false
- Remove every permission string (NSCameraUsageDescription, NSUserTrackingUsageDescription, etc.) UNLESS my spec lists a feature that actually requires it.
- version: 1.0.0 buildNumber: 1

Then run `npx expo-doctor` – must show N/N checks passed.

■ PROMPT 08 Ship to App Store Connect end-to-end (one shot)

Now run the full ship sequence WITHOUT pausing for input:

1. `npx eas build --platform ios --profile production --non-interactive`
Wait for it to finish.
2. `npx eas submit --platform ios --non-interactive` (uses my ASC key).
3. Poll the ASC builds API until the build state is VALID.
4. Attach the build to App Store Version 1.0 via the ASC API.
5. Upload description / keywords / supportURL / marketingURL / promotionalText via the `appStoreVersionLocalizations` PATCH.
6. Upload the 12 screenshots + the App Preview video.
7. Create a `reviewSubmission`, attach the version as a `reviewSubmissionItem` (IAPs auto-bundle), PATCH `submitted:true`.

Print the final `reviewSubmission` state. If anything fails, surface Apple's exact error message – do NOT retry blindly.

■ PROMPT 09 Arm the auto_ship watcher for v1.0.1+

Create `/app/backend/services/auto_ship.py` with these properties:

- Runs every 30 min via `APScheduler` inside the FastAPI worker.
- Polls the ASC API for version 1.0's `appStoreState`.
- When the state becomes `READY_FOR_SALE` or `PENDING_APPLE_RELEASE`, it auto-ships the next version: bump `app.json`, run `eas build`, `eas submit`, attach to version 1.0.1, upload fresh screenshots, submit `reviewSubmission`.
- Idempotent: stores state in `/app/store/.auto_ship_state.json` so it never double-ships.

Confirm the watcher's first tick logged successfully and paste the log line that proves it.

■ PROMPT 10 Final smoke test + hand-off

Run a final regression pass:

- `deep_testing_backend_v2` (all critical endpoints).
- Take a fresh web preview screenshot of every tab.
- `curl GET /api/system/btc_rate` (or your live-data endpoint) and paste the JSON to prove no simulated data is left.
- Show the ASC final state: version, build, IAP states, `reviewSubmission`.

Then write a `/app/HANDOFF.md` that documents:

- every credential the app uses + where it lives
- every cron job + its schedule
- how to invoke the `auto_ship` watcher manually if needed

I want to be able to hand this to ANY future engineer in one file.

4 · IF YOU ONLY READ TWO LINES

The two sentences that would have saved ~70% of the credits on Satoshi Cloud Miner

If you can only send two sentences before letting the agent work, these are the two:

SENTENCE 1

"Before writing a single line of IAP code, enumerate my App Store Connect product IDs via API and tell me the canonical list. I will only use those — no new SKUs invented in the backend."

SENTENCE 2

"Any 'AI', 'rate', 'price', or 'live data' field in the UI MUST be backed by a real provider with a fallback cascade. Show me one curl response per data source proving the wire is live before connecting it to the UI."

The fool-proof header — paste it as the first 8 lines of any iOS-clone prompt

■ PROMPT HEADER FOOL-PROOF HEADER

ROLE: ship-the-app engineer, not interim-demo engineer.

DEFINITION OF DONE: WAITING_FOR_REVIEW in App Store Connect.

CREDS: <paste all keys here, do not ask me later>

HARD CONSTRAINTS:

- Zero simulated data.
- App Store Connect is the source of truth for product IDs.
- Full-bleed RGB icon, no alpha, no white border, pixel-verified.
- supportsTablet:false unless explicitly requested.
- No unused iOS permission strings.
- expo-doctor + tsc must pass before any EAS build.
- Use the ASC API for every upload; no manual upload guides.

EXIT CRITERIA: auto_ship watcher armed for the next version.

5 · TL;DR — THE ONE LESSON

Verify the platform's reality BEFORE writing the code that depends on it

Both of the most expensive bugs on the Satoshi Cloud Miner project — the **starter_099 'Purchase failed'** in TestFlight and the **white-border app icon** — came from the same mistake: writing code based on what we *assumed* the platform / asset-generator had produced, instead of programmatically inspecting what was actually there.

If every future iOS clone starts with a 5-minute '*enumerate the platform reality*' step (list ASC's actual IAPs, sample the icon's actual edge pixels, probe the actual video's audio track), we ship in 1–2 EAS builds. Not 20.

The four reality-checks every iOS clone needs on Day 1

#	Reality check	How (one line)
1	What product IDs exist in App Store Connect?	GET /v1/apps/<id>/inAppPurchasesV2 – list and pin those as your
2	What does the 1024x1024 icon actually look like at the edges?	EdgeSample 10 random pixels within 30px of each edge. Assert n
3	What's the live BTC/FX/whatever rate right now?	curl the provider. Cache. Add 2 fallback providers. No constant
4	What audio track does my preview video really have?	ffprobe -v error -show_streams app-preview.mp4 grep audio – a

Final word

Send the **Master Prompt** (page 3) + the **10 Numbered Prompts** (pages 5–7) in order, and you will be in TestFlight in a handful of EAS builds, not twenty. Keep this PDF next to every new iOS project — it pays for itself the first time you use it.

— end of playbook —